

Informe del Proyecto Integrador - Juego de Trivia por Turnos

Integrantes: Bárbara y Sol (Grupo 7)

Asignatura: Informática I

Categoría / Ambientación: Categoría 3 - Trivia / Quiz por turnos - VideoJuegos

1. Introducción y Descripción del Proyecto

El objetivo de este proyecto fue diseñar e implementar un juego interactivo de preguntas y respuestas (Trivia) que se ejecuta enteramente por consola en lenguaje Java. El juego permite la participación de jugadores por turnos, calculando puntajes y ordenando la tabla de posiciones al finalizar la partida.

Para cumplir con las exigencias de la asignatura, el desarrollo se centró en aplicar los conceptos fundamentales de la Programación Orientada a Objetos (POO): encapsulamiento, uso de estructuras de datos dinámicas, manejo de excepciones personalizadas para el control de errores, y la implementación de un algoritmo de ordenamiento sobre los resultados de los jugadores.

2. Arquitectura y Diseño de Clases (UML)

El sistema se compone de una serie de clases bien encapsuladas que representan las entidades principales de la trivia:

- **Jugador**: Representa a cada usuario que participa en la partida.
 - Atributos: `nombre (String)`, `puntaje (int)`.
 - Responsabilidades: Mantiene el nombre del participante y acumula el puntaje obtenido a medida que responde correctamente las preguntas. Incluye sus respectivos métodos getters y setters para asegurar el encapsulamiento.
- **Opcion**: Representa una alternativa de respuesta para una pregunta determinada.
 - Atributos: `texto (String)`, `esCorrecta (boolean)`.
 - Responsabilidades: Guarda la descripción de la opción y valida si dicha opción corresponde a la respuesta correcta de la pregunta mediante un valor booleano.
- **Pregunta**: Modela una pregunta individual dentro del juego.
 - Atributos: `enunciado (String)`, `opciones (ArrayList<Opcion>)`.
 - Responsabilidades: Almacena el texto de la pregunta y una lista de opciones asociadas. Incluye lógica para mostrar sus alternativas y validar si la respuesta ingresada por el usuario es la correcta.
- **JuegoTrivia** (y **JuegoTriviaFINAL**): Corresponde al controlador principal de la lógica del juego.
 - Atributos: `preguntas (ArrayList<Pregunta>)`, `jugadores (ArrayList<Jugador>)`.

- Responsabilidades: Administra el flujo general del juego: la carga de preguntas, el registro de jugadores, el control de turnos, la evaluación de respuestas y la presentación final de puntajes.
- `RespuestaInvalidaException`: Excepción personalizada que extiende de `Exception`.
 - Responsabilidades: Permite capturar y gestionar los errores cuando el jugador ingresa un valor fuera del rango numérico permitido para las opciones presentadas.
- `Main`: Clase principal que contiene el método `main`. Se encarga de inicializar el menú del juego en consola y dar comienzo a la partida.

3. Decisiones Técnicas y Estructura de Datos

3.1 Estructura de Datos Seleccionada

Decidimos utilizar la clase `ArrayList` de la biblioteca estándar de Java (`java.util.ArrayList`) por las siguientes razones:

1. Flexibilidad: Permite manejar colecciones dinámicas tanto de preguntas como de jugadores sin necesidad de fijar un tamaño estático inicial.
2. Facilidad de recorrido y ordenamiento: Ofrece métodos integrados para acceder recursivamente a los elementos e interactuar fácilmente con los métodos de ordenamiento.

3.2 Manejo de Excepciones

Para garantizar la solidez de la aplicación y evitar cierres inesperados por entradas inválidas en la consola, implementamos la excepción propia `RespuestaInvalidaException`.

- Caso de uso: Si la trivia despliega 4 opciones (1 a 4) y el usuario ingresa un número menor a 1, mayor a 4, o un texto no válido, el sistema lanza la excepción. Se captura la excepción mediante un bloque `try-catch`, mostrando un mensaje explicativo y solicitando nuevamente el ingreso de la opción sin interrumpir la partida.

3.3 Algoritmo de Ordenamiento

Al finalizar la partida, implementamos un algoritmo de ordenamiento (o el uso de `Collections.sort` con un comparador personalizado) sobre el `ArrayList<Jugador>`.

- Propósito: Ordenar la lista de jugadores de forma descendente (de mayor a menor puntaje) para imprimir en pantalla una tabla de posiciones clara con el ganador de la partida en el primer lugar.

4. Conclusión

El diseño implementado cumple con todos los requerimientos funcionales y técnicos exigidos. Al separar responsabilidades entre distintas clases encapsuladas, logramos un código ordenado, legible y fácil de mantener. El uso correcto de excepciones asegura una

experiencia fluida por consola, mientras que la estructura de datos elegida permite procesar y clasificar los resultados finales de manera transparente y eficiente.